Universidade de Aveiro
Departamento de Eletrónica, Telecomunicações e Informática



Information and Coding
2026/27

Lab work n.º 1

Membro	N.º	Participação
Diogo Nascimento	120031	33 3%
Ana Sofia Almeida	113892	33 3%
Beatriz Saraiva	108944	33 3%

October 25, 2026

Contents

1 Part I **2**

2 Part II **3**

3 Part III **5**

Part I

The general use of AI throughout the whole project involved code translation, project architecture design evaluation, report correctness and general concept clarification.

For example, when it came to the simple starting decision of which programming language to use, despite the fact that we already had some understading of matlab and its shortcomings for efficient performance, we started with it just to experiment, and later, with the help of AI (more specifically models like Copilot and Claude), we reached our conclusion that C++ would be the best programming language to work with later on. We were also suggested Rust, but it was seemingly less beginner friendly, and its only advantages included strict but safer thread and memory management, which we were willing to trade off.

For the first part, as we started by learning how to work with WAV files, and how they are structured, for later manipulation, AI was used to help better visualize this file structure. Among this, other problems we faced include:

1. **Translating Matlab Code:** Given our initial prototypes, and to make work start more efficiently, after understading the types of operations done, we used AI models like Gemini to quickly obtain C versions of the files (this was an early mistake, as in a prompt we mistook C for C++, but this error was later remedied).

2. **Makefile:** This was a simply to make a file that would allow the programs to more easily run inside any operating system, and with a more streamlined initiation process.

3. **Creating an UI:** Specifically for the first part, we believed that we needed to have a user interface that allowed a user to interact with the main functionalities developed (histogram plotting, audio effects...). However, even after being clarified that it wasn't necessary, we decided to make it anyways, just to help testing multiple audio samples be more streamlined. Also, during the histogram section, after migrating from matlab, we learned that plotting and displaying the histograms was going to be more difficult in C++, so while trying to figure out how to do so, we took advantage of this UI to help visualize its correctness.

4. **Correcting the Report:** While writing our report, by the end we quickly passed it through Gemini to help identify possible grammatical errors, as well as technical ones. For example, for the format of the WAV files, when listing the chunks, we failed to address that FMT chunks also included audio format, byte rate and block align.

For prompting, the general strategy used included a contextualization phase, where we would give a brief context of our approach for a problem (for example, we would show the code we had for extracting the relevant data from a wav, and an empty function where we would want to create a histogram). Given the context, we would ask for a solution, as well as an explanation of the solution and sometimes, if we had difficulty, employ strategies like analogical explanations (for example, during the beginning, by comparing an audio WAV file to a package with a label, we understood better how the headers and samples worked together).

Part II

For our lossless codec, AI helped us discover the working set. To do this, I had to give it access to the audio files and the codec's build folder, as well as limited access to PowerShell and the command line. The procedure was as follows:

1. It saved each sequence of instructions in the script `"monitor-working-set-clss.ps1"`. It then started it from a PowerShell monitor and waited for the executions.
2. The monitor searched for the processes `wav_lossless_enc.exe`, `wav_lossless_dec.exe`, and `MyLosslessCodec` inside the build folder and recorded the PID (Process Identifier) and the timestamps.
3. For each compression or decompression operation that I started in VS Code, PowerShell read the command line associated with the process to determine the file name and the operation.
4. While each process was running, PowerShell updated the process data approximately every 10 ms and queried `PeakWorkingSet64`, which indicates the largest working set reached by the process up to that moment. To do this, while launching each CLSS process, the script recorded the process data and, at the end, placed it in the file `"working-set-clss.csv"`.
5. Finally, it saved the largest value in bytes and converted it to MiB by dividing it by 1,048,576.

For the other codecs and compressors, however, it carried out the entire procedure. To do this, I once again gave it access to the audio files, PowerShell, and the respective codecs and compressors. In this case, it did not need access to the command line, because, in our codec, the command line was used to start the programs with the file paths. In this case, since AI started the process from PowerShell, it did not need to specify the executable and the files, since PowerShell received that command.

The procedure was as follows:

1. It read the 20 original WAV files in the audio folder.
2. It executed the codec and compressor programs—FLAC, WavPack, TAK, Zstd, XZ/LZMA, and CLSS—through PowerShell. To do this, the file `"benchmark-comparacao.ps1"` started the codecs and compressors, measured them, and recorded the results, logging the times, sizes, working set peaks, configurations, and fidelity checks.
3. It read each WAV file, compressed it, and decompressed it, writing the obtained data to `"benchmark-comparacao.csv"`.
4. In `"benchmark-comparacao-resumo.csv"`, it produces the aggregate summary by codec: compression ratio, times, speed, maximum working set, and the results of the fidelity checks.

AI was a very important tool throughout the development of our lossless codec. In the first phase, we began by researching the main stages involved in lossless audio encoding and decoding. Based on this research, we identified the following workflow: for compression, reading the WAV file → prediction and calculation of the residuals → ZigZag encoding → Rice encoding; for decompression, the reverse process: reading the compressed file → Rice decoding → ZigZag decoding → reconstruction of the residuals and samples → generation of the WAV file.

We sought to deepen our knowledge of each of these stages in order to understand the theoretical concepts and their practical implementation. To do this, we used various sources, including Internet searches, GitHub repositories, and technical documentation, complementing this research with requests for clarification and recommendations from AI.

After a period of research and development, we managed to build our first functional version of the codec. However, when we reached question 6, we found that its performance, in terms of compression, was significantly lower than that of the other codecs analysed. To try to identify the causes of this problem, we submitted the complete code to AI and requested an analysis of the reasons why our solution produced less favourable results.

The analysis allowed us to identify several limitations in the initial implementation. Firstly, we used only a first-order predictor, without exploring potentially more efficient alternatives. Furthermore, we chose a single parameter (k) for the entire file, instead of adapting it to the local characteristics of the signal. Finally, we applied Rice encoding to all the residuals without checking whether it was actually more advantageous than other forms of representation. In the case of stereo audio, we also did not evaluate whether the Mid/Side transformation would make it possible to reduce the amount of information required to represent the signal.

Based on these observations, we introduced improvements to the codec architecture. We began dividing the audio files into blocks, allowing each block to be encoded independently. For each block, the algorithm can choose the most suitable parameter (k) and determine whether it is worthwhile to use Rice encoding or store the samples in uncompressed PCM. In this way, the same file can contain blocks encoded with different parameters (k) and can even combine compressed blocks with blocks stored without compression, depending on which option is more efficient in each case.

Additionally, for stereo files, we introduced an evaluation of the Mid/Side transformation, comparing the original representation of the left and right channels with a representation based on the *Mid* and *Side* signals. The objective is to select the alternative that allows a more compact representation to be obtained, without loss of information.

AI played a relevant role in identifying these limitations and developing the improvements implemented. Nevertheless, we tried not to limit ourselves to accepting its suggestions or copying the code it produced. We attempted to understand the function of each block, assess its suitability for the codec's operation, and verify whether the changes made sense from a theoretical and practical point of view. For this reason, much of the code was also commented on by us, both to document how it works and to make it easier to understand.

In conclusion, we consider AI to have been a valuable tool for accelerating research, supporting problem-solving, and exploring possible improvements. However, its use requires critical thinking and understanding on the part of the person using it. **AI can help us find solutions, but it is up to us to understand, evaluate, and validate those solutions.**

Part III

Part III (a lossy codec based on the DCT and a comparison with MPEG codecs) was developed using Claude Code (Anthropic).

We defined two rules right at the beginning, which shaped the entire interaction:

- The AI writes the code (C++17, CMake); the decisions are ours.
- The work is divided into small steps, and at the end of each step we test it ourselves before moving on.

What It Was Used For

- **Planning:** Dividing the codec into steps that compile and can be tested one at a time.
- **Code:** WAV reading/writing, fast MDCT (FFT), quantizer, Mid/Side, context-based Rice coding, file format, rate control, measurement and testing scripts.
- **Verification:** All automated tests were written by the AI (tools/system_test.sh: system tests with sanitizers and fuzzing); we confirmed the results using our Part I tool.
- **Documentation:** READMEs.

Problems: What the AI Proposed, How It Was Verified, and What Changed

Cases in which the AI's first proposal was wrong or incomplete.

Problem	What the AI Proposed	How It Was Verified	Result
Comparing v1 with v2	Bitrate with the same quantization step	With the same step, the SNR differed by 0.5 dB (deadzone)	Comparison was changed to use the same SNR (BD-rate, Bjøntegaard method): $-32.7\,\%$ in bits
Use of Mid/Side in the README	"Used in 70–99 % of blocks, never in sample02"	Measured using the encoder statistics (-v)	Wrong: 30–96 %, and ∼3 % in sample02. Corrected
RDO Quantization	Started implementing it immediately after the deadzone	We interrupted it: it had not been requested and the step was closed	Left as future work, with the explicit indication "not measured"
Automated Test Criterion	"SNR > 30 dB" with the default step	Failed with our quieter WAVs, even though the codec was working correctly	Incorrect criterion: SNR depends on volume. Replaced with an error limit derived from the step, MSE $\leq (1-r)^2 \cdot q^2/3$, justified by measurements
Test Script Argument	Accepted only a directory	With a file, it said "no WAVs" and skipped 4 groups of tests without warning; the AI tried to guess other causes before seeing the output	Changed to accept either a file or a directory, and to stop with a clear message in other cases
Alignment in the Comparison with MPEG	Add an offset search to wav_snr (MP3/AAC/Opus introduce a delay at the beginning)	We tested this first: with .mp3, .m4a and .opus, ffmpeg already removes the delay (offset 0); only raw .aac was offset by 1024 samples	Simpler solution, without changing src/: use these formats and have the script verify alignment, stopping if it is not 0

How We Verified the Results

No result was accepted simply because the AI said so:

- **Independent Tool:** The SNR measured by wav_snr in the lossy codec was compared with our wav_cmp from Part I, and the values match.
- **Lossless Limit:** With the minimum step, 8-bit and 16-bit files are reconstructed exactly (maximum error 0).
- **System Tests:** All automated tests were written by the AI. There are 82 end-to-end tests, compiled with AddressSanitizer and UndefinedBehaviorSanitizer, including invalid inputs and 200 randomly corrupted compressed files. We also ran them with our own WAVs, which is how we discovered the incorrect test criterion.
- **Reproducibility:** All tables in Report 1 are generated by a script. Running the scripts twice produces the same numbers; only the timings vary, by about 10 %.
- **Listening:** A script generates the encoded files and the codec noise at several bitrates, so that each decision can be heard and not just measured.

Critical Evaluation

What Worked Well

It was most useful where the work was extensive but well-defined: writing the codec and the measurement scripts.

It suggested methods we were not familiar with: BD-rate (used in video coding), ablation to measure each improvement, and sanitizers and fuzzing for testing.

What Did Not Work So Well

- **Confident Claims Before Measuring.** Several statements (use of Mid/Side, size of the file format, duration of the files) were presented as facts and were incorrect. They were only corrected because we required a measurement for every number.
- **Doing More Than Requested.** It started implementing features we had not requested (the RDO quantizer, extra rules in the tests). We had to explicitly ask it to explain each change before applying it.
- **Guessing.** When asked about a failure it had not seen, it gave plausible but incorrect causes. The actual cause became clear as soon as we showed it the output.